

“Raising Exceptions vs. Returning Error Codes”), but you should reserve returning `None` for functions that have no meaningful return value.

The reason is that returning `None` to indicate an error is a common source of uncaught `'NoneType' object has no attribute` exceptions:

```
>>> import random
>>> def sometimesReturnsNone():
...     if random.randint(1, 2) == 1:
...         return 'Hello!'
...     else:
...         return None
...
>>> returnVal = sometimesReturnsNone()
>>> returnVal.upper()
'HELLO!'
>>> returnVal = sometimesReturnsNone()
>>> returnVal.upper()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'upper'
```

This error message is rather vague, and it could take some effort to trace its cause back to a function that normally returns an expected result but could also return `None` when an error happens. The problem occurred because `sometimesReturnsNone()` returned `None`, which we then assigned to the `returnVal` variable. But the error message would lead you to think the problem occurred in the call to the `upper()` method.

In a 2009 conference talk, computer scientist Tony Hoare apologized for inventing the null reference (the general analogous value to Python’s `None` value) in 1965, saying “I call it my billion dollar mistake. [...] I couldn’t resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last 40 years.” You can view his full talk online at <https://autbor.com/billiondollarmistake>.